Milestone 1: Requirement Analysis & Architectural Design

Email Automation & Alerts Platform

---

1. Functional requirements

1. Create one-time, interval, or cron-scheduled emails, with HTML/plain body, recipients, and CC
2. Automatically dispatch each scheduled email at its due time
3. Create URL-monitoring alerts: HTTP status check, keyword present, keyword missing, or response-time threshold
4. Periodically evaluate alert conditions and email recipients when one triggers
5. Enforce a cooldown period so a persistently failing alert doesn't re-notify on every check
6. Pause, resume, edit, or delete any schedule or alert
7. Persist an immutable log of every send attempt and every alert check
8. Expose a REST API for programmatic management
9. Provide a dashboard UI so non-technical users can manage everything without the API
10. Recover all active schedules automatically after a service restart

2. Non-functional requirements

- **Reliability** — a job must not be lost if a worker crashes mid-task (durable queue, at-least-once delivery)
- **Scalability** — worker count scales horizontally as job volume grows, independent of the API
- **Timeliness** — jobs dispatch within a bounded delay of their due time (target: under 30 seconds)
- **Idempotency** — a retried job must never send a duplicate email
- **Availability** — target 99.5%+ uptime for the API and scheduler
- **Security** — authenticated API access; SMTP credentials encrypted at rest and never logged
- **Observability** — structured logs and queryable history for every send and check
- **Auditability** — logs are append-only and traceable to the job or alert that produced them
- **Maintainability** — clear boundaries between API, scheduling, workers, and delivery, so each can change independently

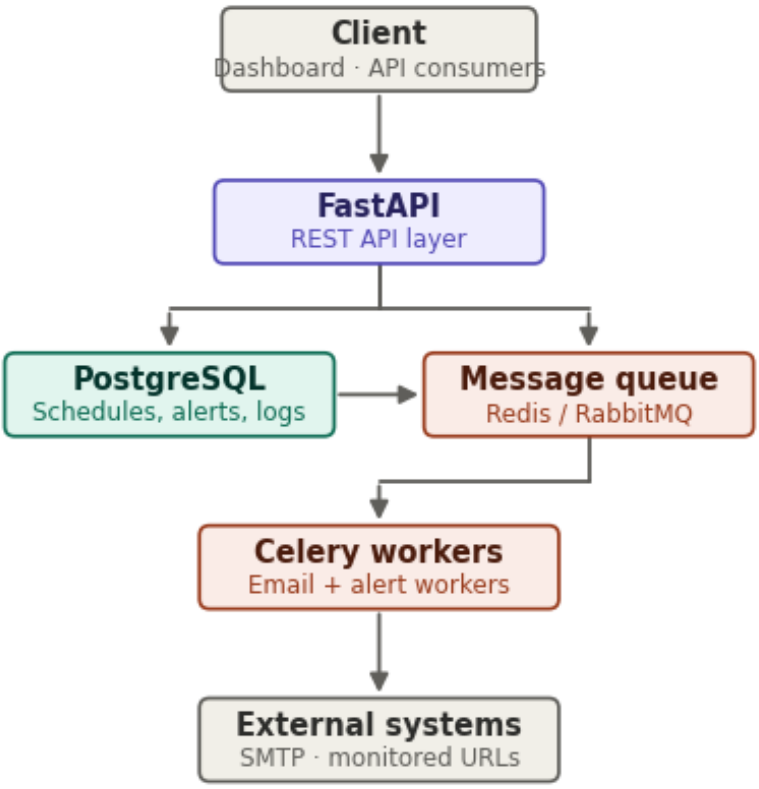
3. Technology stack

Layer	Choice	Why
API	FastAPI + Pydantic	Async, auto-generated docs, strong input validation

Layer	Choice	Why
Data store	PostgreSQL (SQLite for local dev)	ACID guarantees for schedule and log integrity
ORM	SQLAlchemy	Mature; runs unchanged against both databases
Message queue	Redis or RabbitMQ	Durable broker that decouples the API from execution
Task execution	Celery + Celery Beat	Beat triggers due jobs on schedule; workers execute them and scale independently
Email delivery	smtplib, or SES / SendGrid in production	Swappable delivery backend behind one interface
Dashboard	Streamlit	Fast internal tool, no separate frontend build
Deployment	Docker Compose (dev) → Kubernetes (scale)	Workers scale separately from the API
Monitoring	Flower + structured logs	Visibility into queue depth and worker health

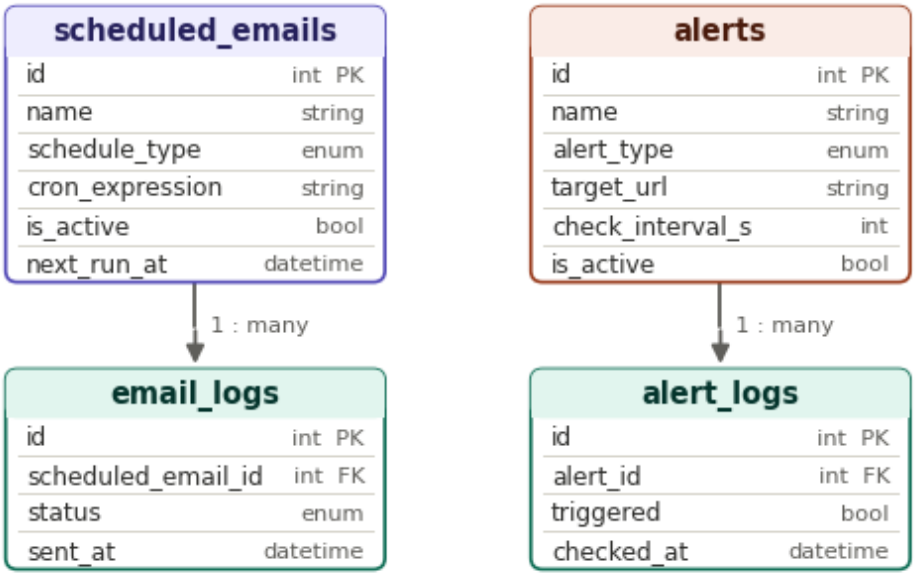
Note: this is the target architecture for scale. The codebase already delivered uses in-process APScheduler — simpler, single-process, no external broker. The message queue is the upgrade path once job volume or reliability needs outgrow one process.

4. System architecture



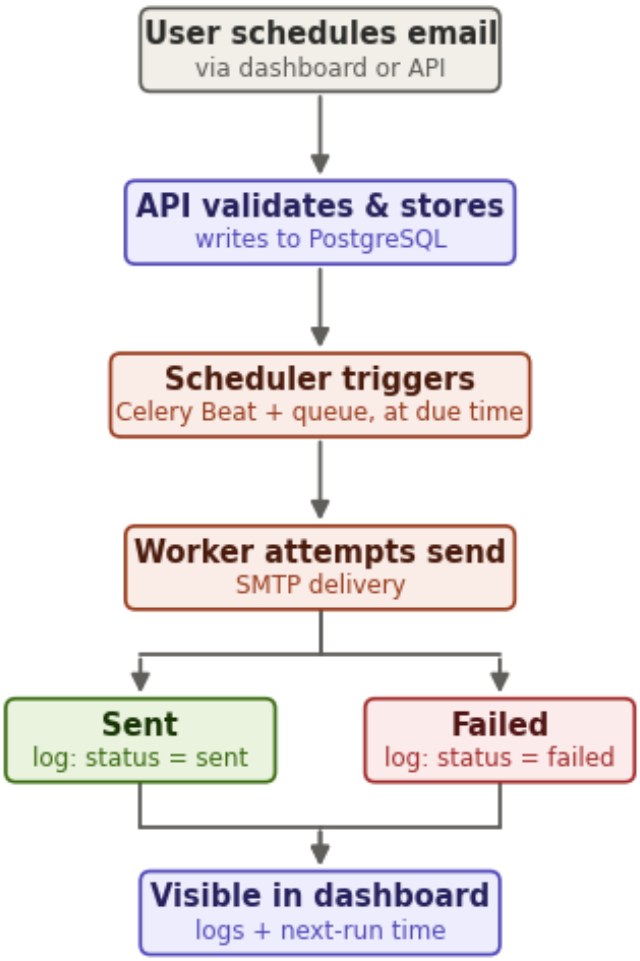
Client requests flow through FastAPI to PostgreSQL. PostgreSQL feeds due jobs to the message queue (FastAPI can also enqueue ad-hoc jobs directly, e.g. a manual “run check now”). Celery workers consume the queue and reach external systems (SMTP, monitored URLs), then write results back to PostgreSQL as logs.

5. Database schema



Two core entities (scheduled_emails, alerts) each have a one-to-many log table capturing every execution attempt, so history is preserved even after a schedule or alert is edited or deleted.

6. User interaction flowchart



From the moment a user schedules an email to the moment the result is visible in the dashboard, every step is logged — including which branch (sent or failed) was taken.

7. Open items for the next milestone

- No auth on the API yet — needs an API key or OAuth layer before internet-facing deployment
- Move from in-process APScheduler to Celery + Redis/RabbitMQ for horizontal scaling
- No automatic retry on failed sends — logged but not requeued